



PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA

MALTA

Machine Learning Theory
and Applications Lab

Aprendizado Profundo II

Parameter-Efficient Fine-Tuning

Prof. Me. Otávio Parraga

Roteiro

- Mixed Precision
- Multi-GPU Training
- LoRA

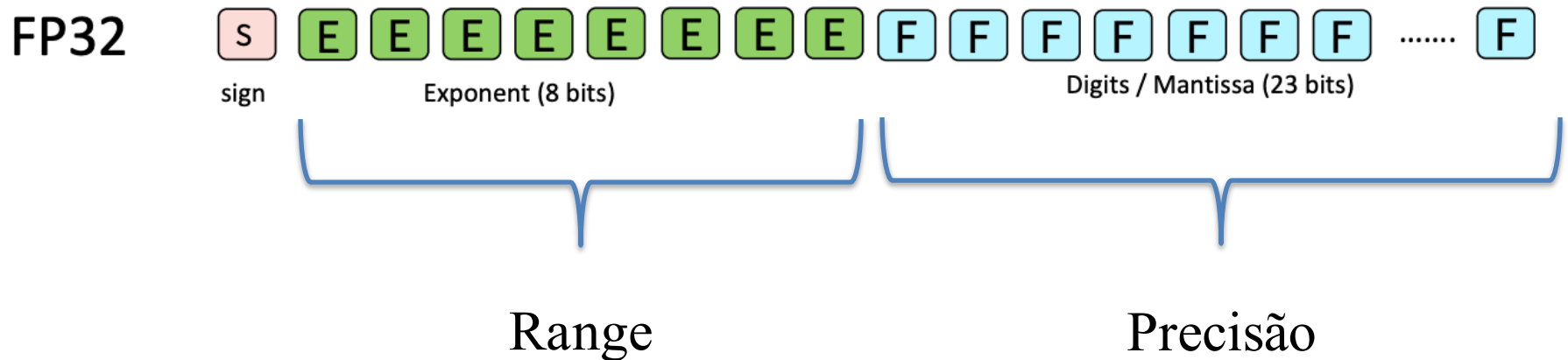
FP32 x FP16

- Tipo de dado que geralmente usamos é **Float**
32 -> FP32



FP32 x FP16

- Tipo de dado que geralmente usamos é **Float**
32 -> FP32



FP32 x FP16

- Tipo de dado que geralmente usamos é **Float**
32 -> FP32
- Podemos usar como alternativa **FP16**



Mixed Precision

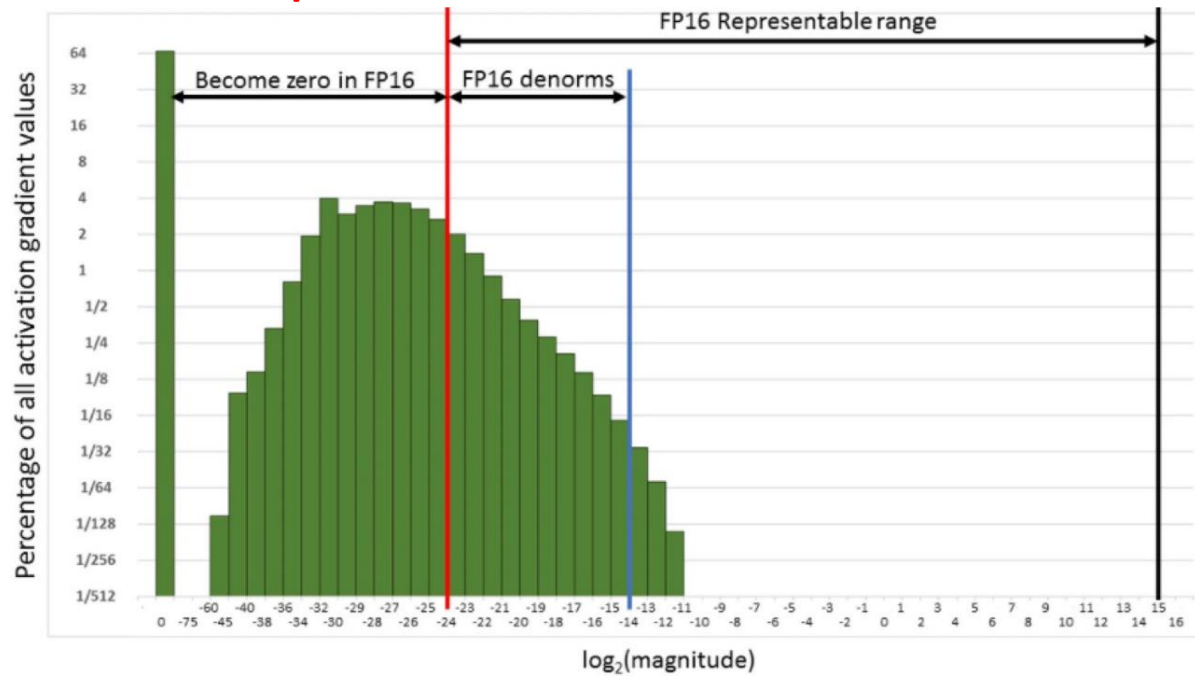
- Se alocamos modelos grandes, podemos ter **erros de memória** durante treinamento
 - Famoso CUDA OOM
- Além disso, computações com **FP16 são muito mais rápidas** em hardware moderno
- Possível alternativa, usar FP16!

Mixed Precision

- Se alocamos modelos grandes, podemos ter **erros de memória** durante treinamento
 - Famoso CUDA OOM
- Além disso, computações com **FP16 são muito mais rápidas** em hardware moderno
- Possível alternativa, usar FP16!
- Porém:
 - Menor range de valores
 - Menor precisão

Mixed Precision

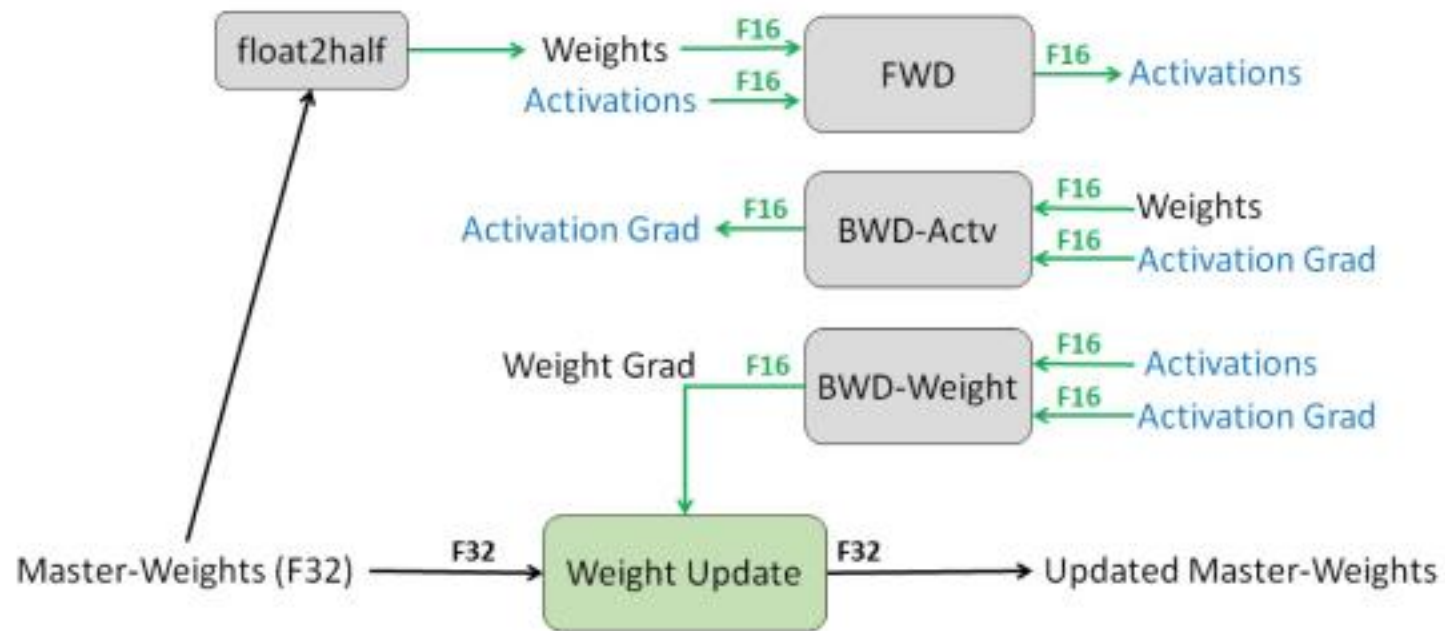
- Porém:
 - Menor range de valores
 - Menor precisão
 - Gradientes podem desaparecer



Mixed Precision

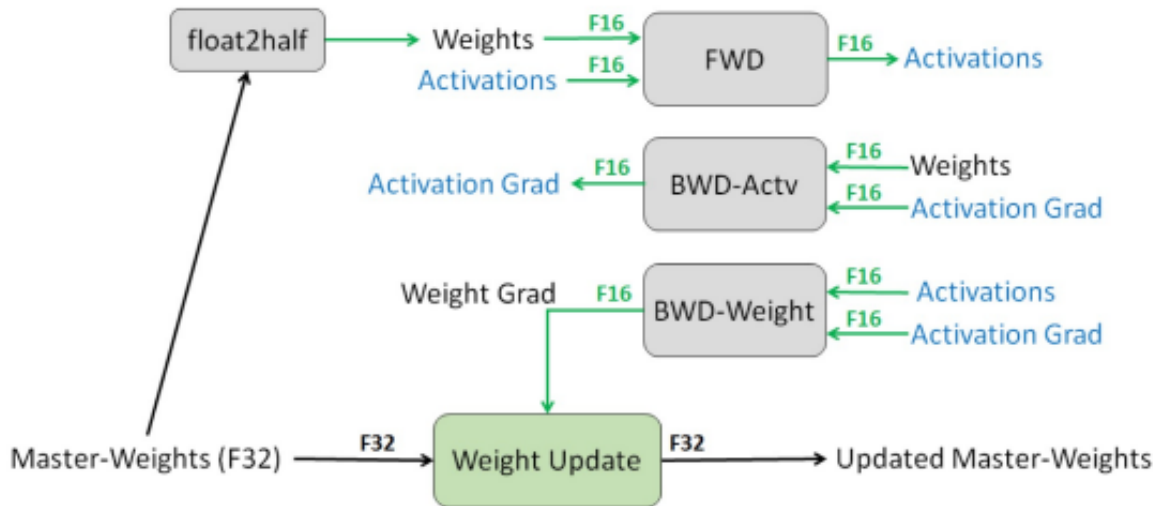
- Usar esses dois tipos de dado é o que chamamos de **Mixed Precision**:
 - Passagens forward e calculo do gradiente em **FP16**
 - Atualização dos pesos em **FP32**
- Cuidado com MP!
 - Como o range de valores é menor, podemos precisar **escalar a loss** por um valor para aumentar os gradientes e impedir que eles zerem

Mixed Precision



Mixed Precision

Sharang et al. 2018

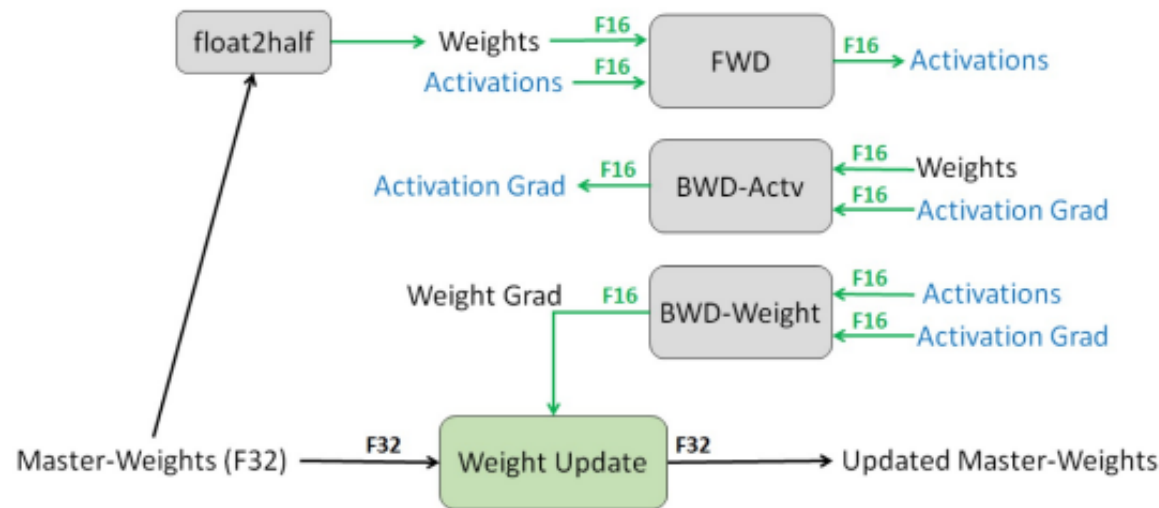


1. Mantemos o modelo em **FP32 (Master Weights)**
2. Forward em **FP16**
3. Backward em **FP16**
4. Cast dos gradientes para **FP32**
5. Atualização dos pesos em **FP32**

Problema! Gradientes podem se tornar 0 por conta do range de valores (underflow)!

Mixed Precision

Sharang et al. 2018

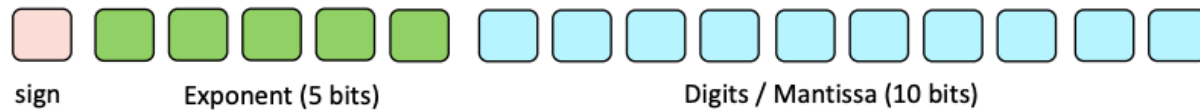


1. Mantemos o modelo em **FP32 (Master Weights)**
2. Forward em **FP16**
3. Multiplicamos a loss por um grande escalar (ex 1024)
4. Backward em **FP16**
5. Cast dos gradientes para **FP32** e dividimos pelo escalar
6. Atualização dos pesos em **FP32**

Mixed Precision

- Alternativa: BrainFloat16

FP16



FP32



BFloat16

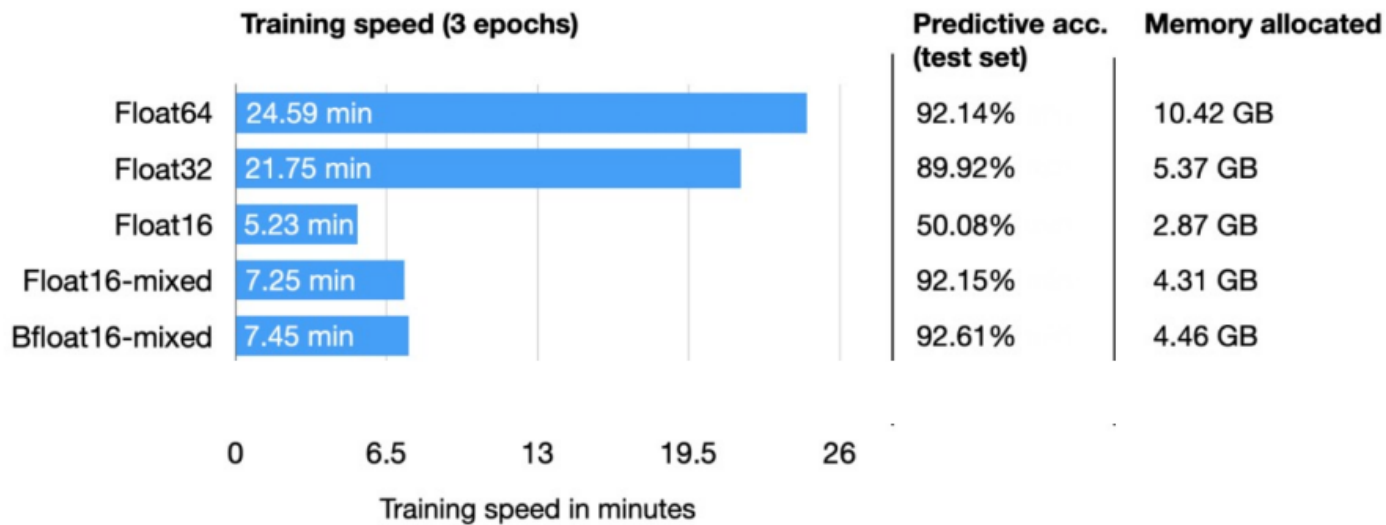


Mixed Precision

- Alternativa: BrainFloat16
- Não precisamos mais de Scaling da Loss

Tipo	Min	Max
float16	-65504	65504
float32	-3.40282e+38	3.40282e+38
bfloat16	-3.8953e+38	3.8953e+38

Mixed Precision



Results from finetuning DistilBERT for sentiment classification on a single A100 GPU.

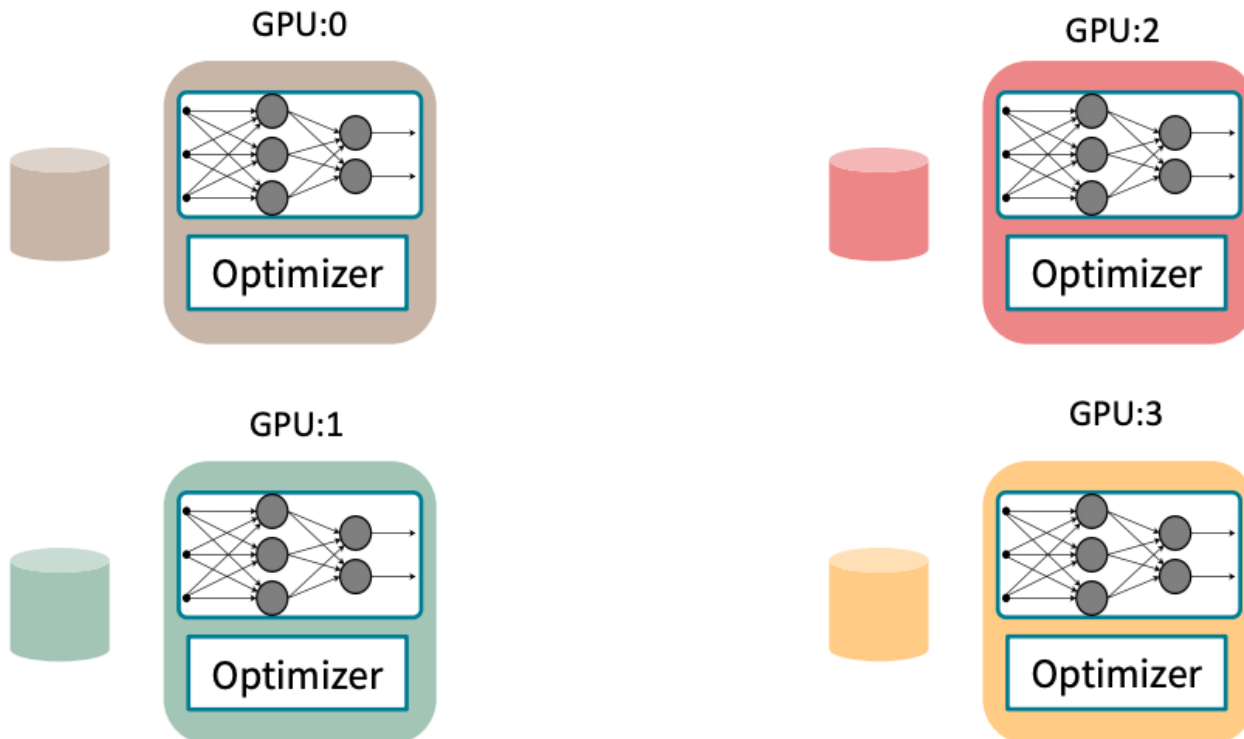
Source: <https://sebastianraschka.com/blog/2023/llm-mixed-precision-copy.html>

Multi-GPU

- Ao treinar um modelo, o que alocamos na VRAM da placa de vídeo?
 - Pesos do modelo durante inferência (FP16)
 - Pesos do Master Weight (FP32)
 - Adam Momentum (FP32)
 - Adam Variance (FP32)

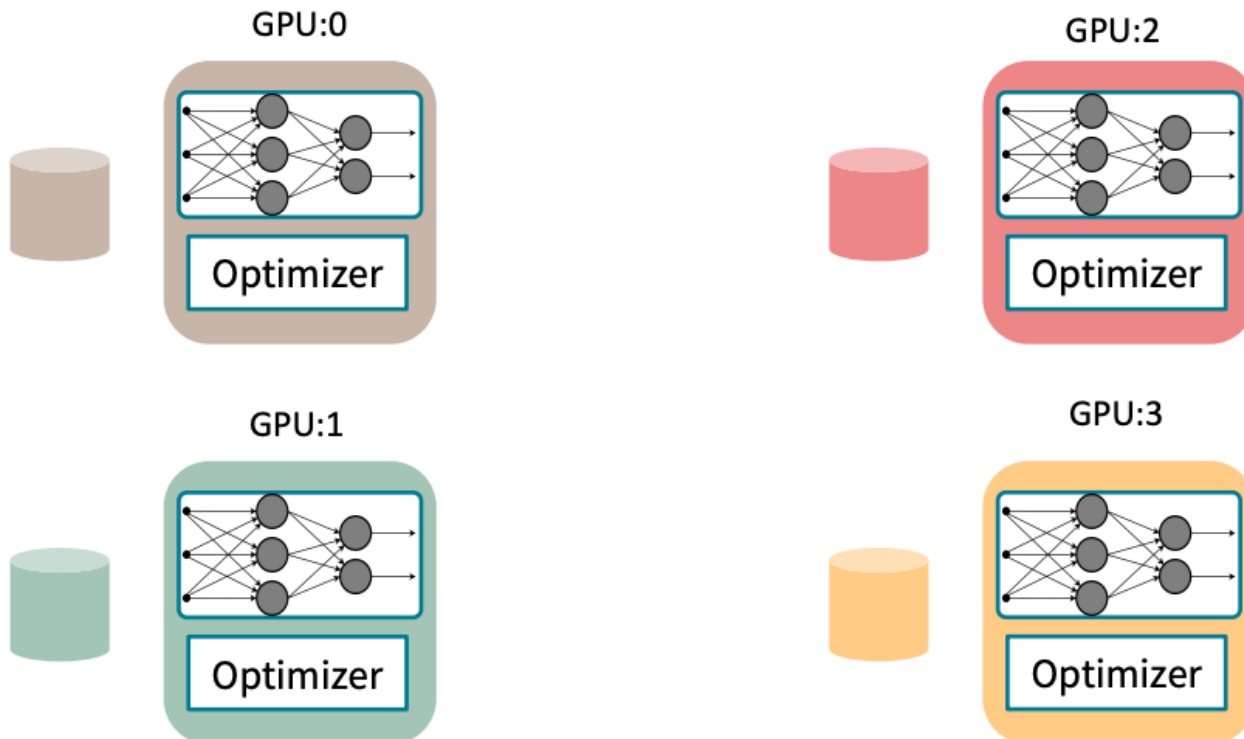
Multi-GPU

- Distributed Data Parallel (DDP)
 - Cada GPU tem uma **cópia sincronizada** do modelo com seu próprio batch de dados



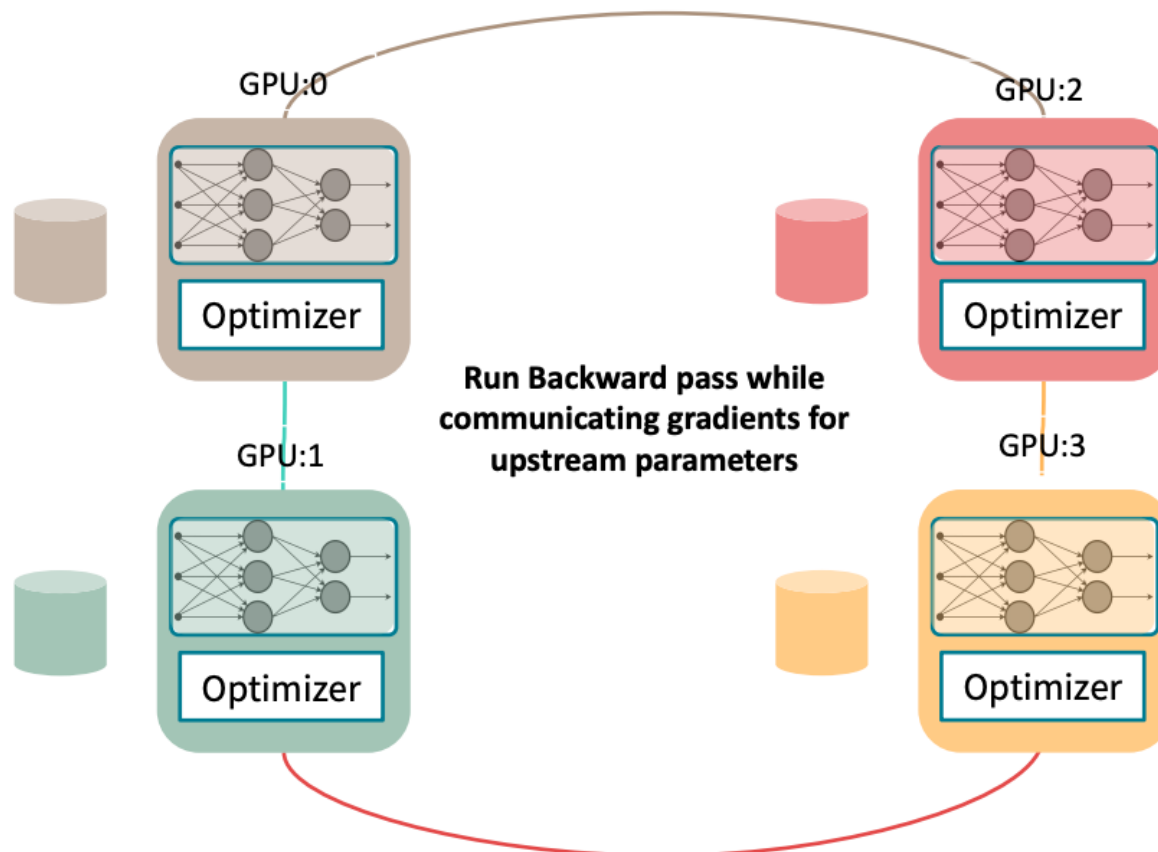
Multi-GPU

- Forward é feito de forma paralela



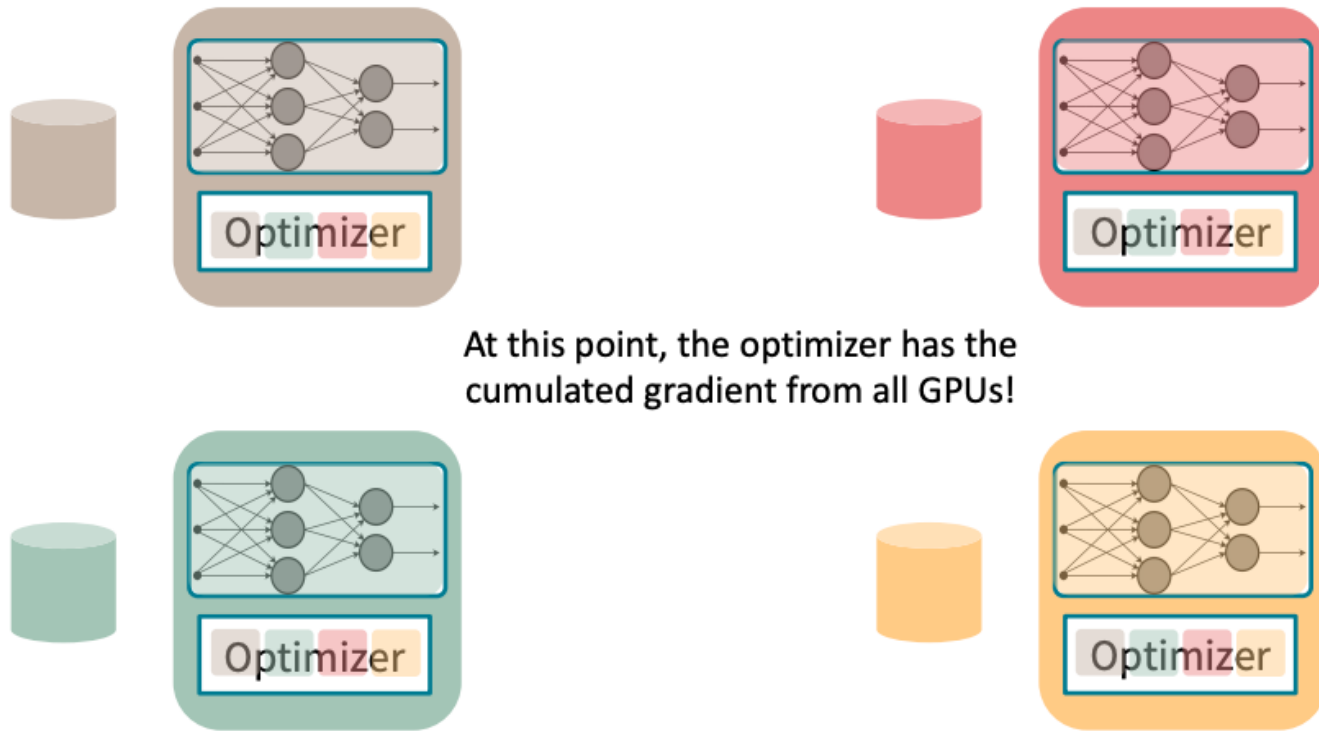
Multi-GPU

- Backward pass é calculado paralelamente e compartilhado com as demais GPUs



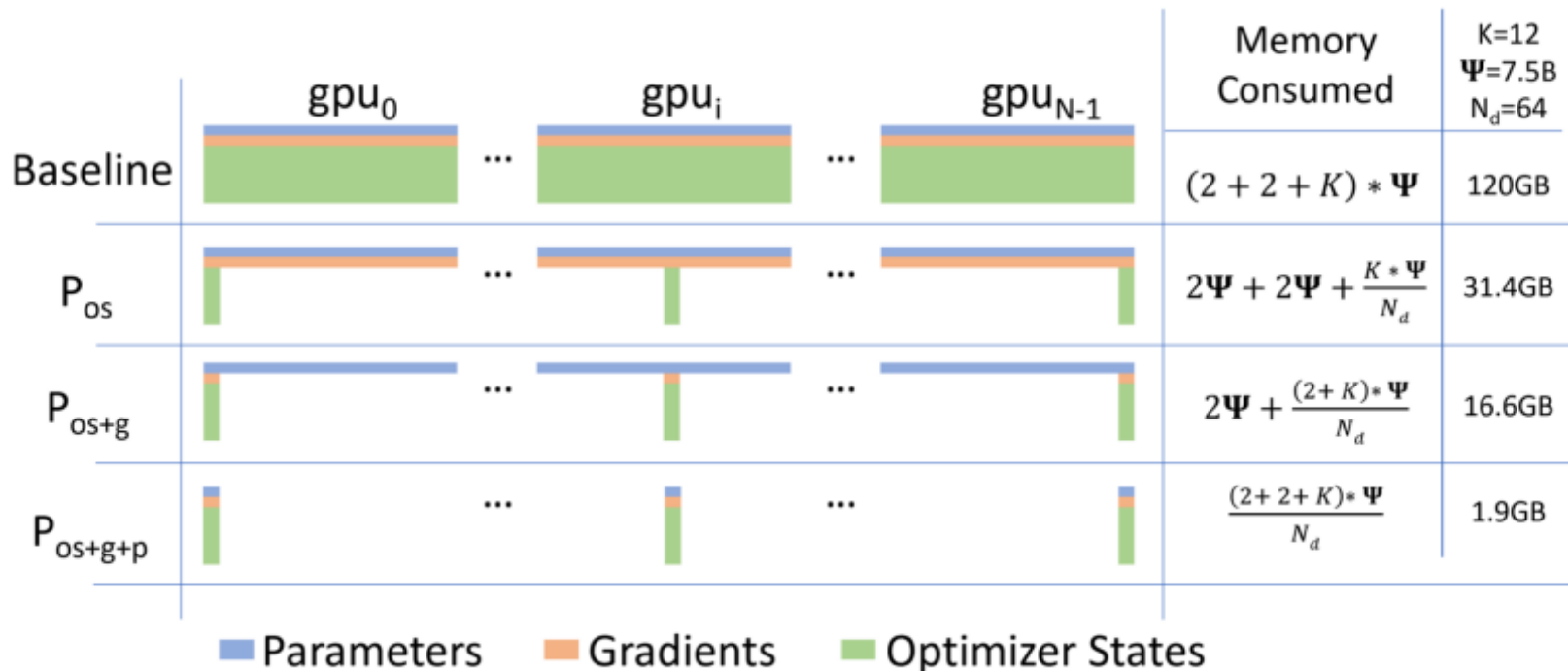
Multi-GPU

- Gradientes são acumulados, atualizamos todos os modelos de forma sincronizada



Multi-GPU

- Problema: DDP normal tem **péssima gestão de memória**
 - Alternativa: ZeRO Stages



LoRA

- Vamos assumir que temos um LLM $P_W(y|x)$
- Se quisermos fazer o **fine-tuning** do modelo para uma tarefa específica, utilizaremos CLM

$$\max_W \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_W(y_t|x, y_{<t}))$$

- Dessa forma, atualizamos W para $W + \Delta W$

LoRA

- Se considerarmos modelos grandes como um Llama 3.1
 - $|\Delta W| = |W|$
 - $|W| = 405B$ no maior dos modelos
- Se formos tentar ajustar vários modelos para **tarefas distintas**, teremos problemas em armazenar e fazer deploy desses modelos

LoRA

- Nossa ideia será a seguinte:
 - Vamos isolar os pesos da tarefa específica a ser aprendida
 - Vamos transformar os pesos para que sejam uma decomposição de matrizes $\Delta W = \Delta W(\Theta)$, onde $|\Theta| \ll |W_0|$

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_{W_0 + \Delta W(\Theta)}(y_t | x, y_{<t}))$$

LoRA

- “Atualizações nos pesos possuem um *low intrinsic rank* durante a atualização”
- $W_0 \in \mathbb{R}^{d \times k}$: Matriz de pesos pré-treinada
- Transformamos a atualização dos pesos em uma decomposição low-rank

$$W_0 + \Delta W = W_0 + \alpha B A$$

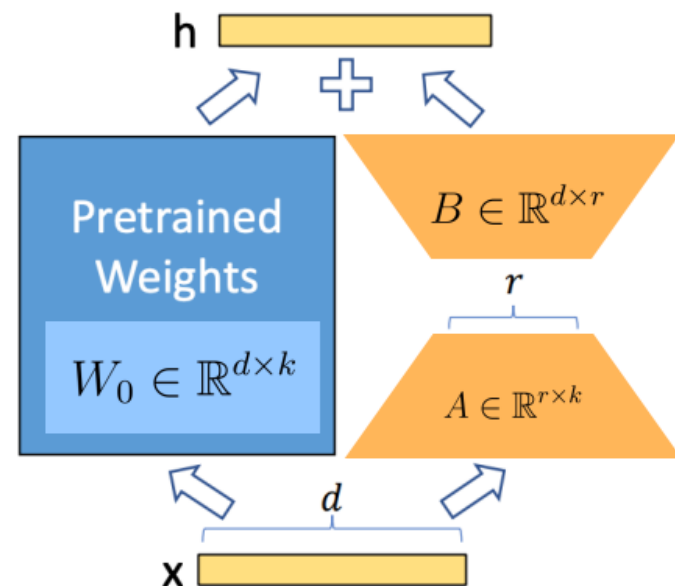
LoRA

- Low-Rank Adaptation (LoRA)
- Transformamos a atualização dos pesos em uma decomposição low-rank

$$W_0 + \Delta W = W_0 + \alpha BA$$

onde $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ e
 $r \ll \min(d, k)$

- α é o “tradeoff” entre conhecimento do pré-treino e conhecimento do fine-tuning
- Apenas A e B são treináveis



LoRA

```
from transformers import AutoModelForImageClassification, TrainingArguments, Trainer

model = AutoModelForImageClassification.from_pretrained(
    "google/vit-base-patch16-224-in21k",
    label2id=label2id,
    id2label=id2label,
    ignore_mismatched_sizes=True,
)

from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=["query", "value"],
    lora_dropout=0.1,
    bias="none",
    modules_to_save=["classifier"],
)

model = get_peft_model(model, config)
model.print_trainable_parameters()

"trainable params: 667,493 || all params: 86,543,818 || trainable%: 0.7712775047664294"
```

LoRA

```
from peft import PeftConfig, PeftModel
from transformers import AutoImageProcessor
from PIL import Image
import requests

config = PeftConfig.from_pretrained("stevhliu/vit-base-patch16-224-in21k-lora")
model = AutoModelForImageClassification.from_pretrained(
    config.base_model_name_or_path,
    label2id=label2id,
    id2label=id2label,
    ignore_mismatched_sizes=True,
)
model = PeftModel.from_pretrained(model, "stevhliu/vit-base-patch16-224-in21k-lora")

url = "https://huggingface.co/datasets/sayakpaul/sample-datasets/resolve/main/beignets.jpeg"
image = Image.open(requests.get(url, stream=True).raw)
image
```

LoRA

- Temos ainda QLoRA
 - Utiliza um outro tipo de dado: 4-Bit NormalFloat
 - Otimizado para quantizar distribuições normais
 - Double Quantization
 - Quantiza as constantes do processo de quantização
 - Paginação dos otimizadores

LoRA

- Na prática:
 - Modelos de 33B podem ser fine-tunados em 24Gg de VRAM
 - Modelos de 65B podem ser fine-tunados em 46Gb de VRAM

LoRA

- Exemplo com QLoRA sem paginação

```
import torch
from transformers import BitsAndBytesConfig, AutoModelForCausalLM
from peft import LoraConfig, get_peft_model

nf4_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)

base_NF4_model = AutoModelForCausalLM.from_pretrained(
    "openlm-research/open_llama_3b", quantization_config=nf4_config
)

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules="all-linear",
    lora_dropout=0.1,
    task_type="CAUSAL_LM",
    bias="none",
)

peft_model = get_peft_model(base_NF4_model, lora_config)
```

Referências

- CS 224 Stanford,
<https://web.stanford.edu/class/cs224n/>
- Blogpost HuggingFace:
<https://huggingface.co/blog/4bit-transformers-bitsandbytes>